



WOLLO University

Department of software engineering

Blueprint based exit material for

Mobile application development

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

❖ Lifecycle method of android application

- onCreate – called when activity first created
 - is the first method
 - responsible for initializing the activity and setting up the UI
- Onstart – called when activity is visible to the user
 - This is the point where the activity is ready to interact with the user.
- Onpause – This can happen when another activity is launched on top of the current activity.
 - The activity loses focus but is still visible to the user
- Onresume - called when activity start interacting to user
 - This method is called when the activity is in the foreground and has focus.
 - The activity is fully interactive, and the user can interact with the UI.
- Onstop – when the activity no longer visible to the user
 - This can happen when the user navigates to another activity or when the activity is destroyed
- onRestart - called after the activity stopped, prior to start
 - activity is stopped and then started again
 - This can happen when the user navigates back to the activity
- OnDestroy - called before the activity destroy
 - This method is responsible for releasing any resources that were allocated by the activity.
 - It is the final method

An Android activity goes through seven major lifecycle stages or callbacks. These are:

- onCreate() - This method is called when the activity is first created. It is the first lifecycle method that is called, and it is where you should do your initial setup, such as inflating your layout and initializing your views.
- onStart() - This method is called when the activity is made visible to the user. This happens after the activity has been created, but before it has been brought to the foreground.
- onResume() - This method is called when the activity becomes the foreground activity. This means that it is the activity that the user is currently interacting with.
- onPause() - This method is called when the activity is no longer the foreground activity. This happens when another activity is brought to the foreground, or when the activity is put into the background.
- onStop() - This method is called when the activity is no longer visible to the user. This happens when the activity is put into the background, or when it is destroyed.
- onRestart() - This method is called when the activity is brought back to the foreground after having been stopped.
- onDestroy() - This method is called when the activity is destroyed. This happens when the activity is no longer needed, or when the system needs to reclaim memory.

In addition to these seven main lifecycle methods, there are also a number of other lifecycle methods that you can override. These methods are called in response to specific events, such as when the user rotates the device or when the activity is sent a broadcast.

The Android lifecycle is an important concept to understand, as it allows you to control how your activity behaves as it goes through different states. By overriding the appropriate lifecycle methods, you can ensure that your activity is properly initialized, saved, and destroyed.

❖ Major component of android application

- Activity – dedicate or manage UI and handle user interaction to the smart phone screen
 - Single screen with user interface
 - Action on screen
 - It displaying UI elements and it extended by user classes
- Service – handle background processing associated with an application
 - Perform long running operation
 - By using service, we can make inter-process communication (IPC).
 - Type of service
 - a) Start service – (startService()) service can continue to run in the background
 - b) Bound service – Works by binding a service to an application using bindService()
 - c) Background service – performs an operation that isn't directly noticed by the user
 - d) Foreground service – performs some operation that is noticeable to the user.
- Broadcast receiver – handle communication b/n android OS and application
 - Respond to broad cast message from other application/from system
 - a dormant component of android that listens to system-wide broadcast events or intents.
- Content provider – handle data and database management issue
 - Supply data from one application to other on request
 - manage access to data stored within the application

setContentView() method is used to set the user interface (UI) of an Activity

Additional component

- Fragment - represent the portion of user interface in the activity
- Views – drawn on screen including button, list forms etc
- Layout – control screen format and appearance of views
- Intent - message wiring component together
- Resource – external element such as string, constant draw able picture
- Manifest – configuration file for application

The major components of an Android application are:

- Activities are the primary entry points for users into your application. They represent a single screen in your application's user interface.
- Services are background components that can run independently of the user interface. They are often used to perform long-running tasks, such as downloading files or playing music.
- Broadcast receivers are components that receive system-wide broadcasts. They can be used to respond to events such as the arrival of a new SMS message or the change in the device's orientation.
- Content providers are components that provide access to data. They are often used to share data between applications.

❖ Event handling in android

- Event handling
 - It refers to the process of detecting and responding to user interactions with the UI.
 - Is done through the use of event listeners and event handlers.
 - Event listeners are objects that are attached to UI components, such as buttons and text fields. When a user interacts with a UI component, the listener detects the event and triggers the appropriate event handler
 - Event handlers are methods that are defined in the code and are responsible for responding to specific events. E.g. `onClickListener( context )`

In Android, there are several types of events that can be handled, including:

1. Click events: These are triggered when a user clicks a button or other UI component.
2. Touch events: These are triggered when a user touches the screen, such as swiping or scrolling.
3. Key events: These are triggered when a user presses a key on the device's keyboard.
4. Focus events: These are triggered when a UI component gains or loses focus.

Event handler	Event listener	Description
<code>onClick()</code>	<code>OnClickListener()</code>	The user either clicks or touches or focuses upon any widget like button, text, image etc
<code>onFocusChange()</code>	<code>OnFocusChangeListener()</code>	When the widget loses its focus ie. user goes away from the view item
<code>onKey()</code>	<code>OnFocusChangeListener()</code>	When the user is focused on the item and presses or releases a hardware key on the device
<code>onTouch()</code>	<code>OnTouchListener()</code>	when the user presses the key, releases the key, or any movement gesture on the screen
<code>onMenuItemClick()</code>	<code>OnMenuItemClickListener()</code>	called when the user selects a menu item
<code>onCreateContextMenu()</code>	<code>onCreateContextMenuListener()</code>	when the context menu is being built(as the result of a sustained "long click")

- Event listener can be registered into way
 - Inside the xml layout code
 - By implementing event listener interfaces (in the java code)

❖ Basic graphics and multimedia support in android

- Some of the key features of graphics and multimedia support in Android include:
 - a. 2D and 3D graphics:
- Android provides support for both 2D and 3D graphics, with libraries such as OpenGL ES and Canvas. The graphics package(`android.graphics`) supports such things as bitmaps (which hold pixels), canvases (what your draw calls draw on), primitives (such as rectangles and text), and paints (which you use to add color and styling).
- Drawing graphics is possible in two ways:
 - a) Using java's `ShapeDrawable` class Use `ShapeDrawable` class to support including shape , then use `Shape` which support general shape types, or use `RectShape()`, for

```
rectangle shape etc. ShapeDrawable mDrawable = new ShapeDrawable();  
this.mDrawable = new ShapeDrawable(new RectShape());
```

- b) Using xml - With Android XML drawable shapes, the default is a rectangle, but you can choose a different shape by using the type tag and selecting the value oval, rectangle, line, or arc. To use your XML shape, you need to reference it in a layout.
- b. Image and video playback: Android provides support for playing back images and videos in a variety of formats, including JPEG, PNG, and MP4.

Use the method called `takePicture(Camera.ShutterCallback shutter, Camera.PictureCallback raw, Camera.PictureCallback jpeg)` for taking images. Playing a video is slightly more complicated than playing audio with the MediaPlayer API, in part because you have to provide a view surface for your video to play on. Android has a `VideoView` widget that handles that task for you; you can use it in any layout manager.

- c. Audio playback: Android provides support for playing back audio in a variety of formats, including MP3 and AAC.

Here are some of the most important APIs for basic graphics and multimedia support in Android:

- Canvas - The Canvas API is used to draw images on the screen. You can use the Canvas API to draw shapes, text, and images.
- Paint - The Paint API is used to configure the appearance of the objects that you draw with the Canvas API. You can use the Paint API to set the color, style, and size of the objects that you draw.
- MediaPlayer - The MediaPlayer API is used to play audio and video files. You can use the MediaPlayer API to play music, movies, and other audio and video files.
- SurfaceView - The SurfaceView API is used to create a custom view that can be used to draw graphics and play multimedia content.

❖ Basic skill of using IDE(Android studio) and SDK for implementing android application

➡ IDE(integrated development environment) Android studio environment

- The `AndroidManifest.xml` file. Used for specifying information about the app for the Android runtime environment. The template you choose creates this file
- The `java` folder. This folder includes activities, tests, and other components in Java source code. Every activity, service, and other component is defined as a Java class, usually in its own file. The name of the first activity (screen) the user sees, which also initializes app-wide resources, is customarily `MainActivity`.
- The `res` folder. This folder holds resources, such as XML layouts, UI strings, and images. An activity usually is associated with an XML resource file that specifies the layout of its views. This file is usually named after its activity or function
- The `build.gradle (Module: App)` file. This file specifies the module's build configuration. The template you choose creates this file, which defines the build configuration, including the `minSdkVersion` attribute that declares the minimum version for the app, and the `targetSdkVersion` attribute that declares the highest (newest) version for which the app has been

optimized. This file also includes a list of dependencies, which are libraries required by the code — such as the AppCompat library for supporting a wide range of Android versions.

➡ **SDK(software development kit) environment**

- SDK is a set of development tools that are used to develop applications for the Android platform. Whether you create an application using Java, Kotlin or C#, you need the SDK to get it to run on any Android device. Google for the Android platform
 - To run SDK by the following step
- Click **tools->Android-> SDK Manager**
- **Android SDK Features**
 - Offline Mapping
 - Dynamic Markers
 - Improvised API compatibility
- **SDK tool**
 - The Android SDK Manager is a tool from Google that lets you install new (and old) versions of the Android SDK
 - This includes a complete set of development and debugging tools for Android. SDK tools are also included with Android Studio.
 - are generally platform-independent and they are required no matter which Android platform you are currently working on. There are a set of tools that get installed automatically when you install Android Studio.

Tool	Description
Android	This tool lets you manage the AVD (Android Virtual Device), projects, and the installed components of the SDK.
Emulator	It lets you test your applications without using a physical device.
Proguard	This tool helps in shrinking, optimizing, and obscures your code by removing unused code.
ddms	It lets you debug your Android applications
Android Debug Bridge (Adb)	This is a versatile command-line tool that helps you communicate with an emulator instance or connected Android-powered device.